



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico

El subtitulo adecuado

August 30, 2026

Algoritmos y Estructuras de Datos

Java con Mates

Integrante	LU	Correo electrónico
Ayala, Ignacio	196/25	ayalanacho11@gmail.com
Della Bonzana, Lara	563/25	dellabonzanalara@gmail.com
Morales, Santiago	481/25	santumorales11@gmail.com
Vannelli, Tiziana	945/25	tizivannelli@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

<http://www.exactas.uba.ar>

1 Macros para escribir en LaTeX

Para facilitar la escritura del trabajo práctico en LaTeX, les proveemos de varias macros que les permitirán escribir especificaciones en lógica de primer orden. Por ejemplo, podrán hacer uso de comandos que definen los conectores lógicos: $\wedge, \vee, \rightarrow$ y también los operadores de la lógica trivaluada $\wedge_L, \vee_L, \rightarrow_L$.

Además, disponen de comandos para cuantificadores: $(\forall x : \mathbb{Z}) (x \geq 0 \vee x < 0)$, $(\exists x : \mathbb{Z}) (x \geq 0)$. A estos comandos se les puede agregar un **Largo** al final para que ocupen múltiples renglones.

2 Ejemplos de uso de las macros

```
TAD EjemploDeTAD {
  obs secuenciaDeCaracteres : seq⟨char⟩
  obs conjuntoDeTuplas : conj⟨⟨nombre : seq⟨char⟩, apellido : seq⟨char⟩⟩⟩

  proc crearTAD(in caracteres : seq⟨char⟩, in secuenciaDeNombres : seq⟨⟨nombre : seq⟨char⟩, apellido :
  seq⟨char⟩⟩⟩) : EjemploDeTAD {
    requiere { predicadoUnilinea(caracteres) }
    asegura { res.secuenciaDeCaracteres = caracteres }
    asegura {
      |res.conjuntoDeTuplas| = |secuenciaDeNombres|  $\wedge$ 
      predicadoMultilinea(secuenciaDeNombres, res.conjuntoDeTuplas)
    }
  }
  pred predicadoMultilinea(s : seq⟨char⟩, conjunto : conj⟨⟨seq⟨char⟩, seq⟨char⟩⟩⟩) {
     $(\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L s[i] \in c)$ 
  }
  pred predicadoUnilínea(s : seq⟨char⟩) { |s| > 0 }
  aux unAuxiliar(k :  $\mathbb{Z}$ , l :  $\mathbb{Z}$ ) :  $\mathbb{Z} = |k - l|$ 
}
```

```

    Usuario ES  $\langle ID : \mathbb{Z}, millasTotales : \mathbb{Z}, millasDisponibles : \mathbb{Z}, historialTransacciones : seq\langle, \rangle factorCategoria : \mathbb{Z} \rangle$ 
    Promocion ES  $\langle nombre : char, factor : \mathbb{Z}, estado : bool \rangle$ 
    TAD AirMiles {
        esto hay que verlo, no hace falta obs promociones :  $seq\langle\langle char, \mathbb{Z}, bool \rangle\rangle$ 
        obs usuarios :  $conj\langle Usuario \rangle$ 
        obs promociones :  $conj\langle Promocion \rangle$ 
        proc crearSistema() : AirMiles {
            requiere { True }
            asegura {  $(\exists p : Promocion) (p \in res.promociones \wedge nombrePromocion(p, res) = 'NoPromo' \wedge factorPromocion(p, res) = 1)$  }
            asegura {  $res.usuarios = \emptyset$  }
            asegura {  $\#res.promociones = 1$  }
        }
        proc registrarUsuario(inout a : AirMiles, in id :  $\mathbb{Z}$ ) {
            requiere {  $!existeUsuario(id, a) \wedge a = A0$  }
            asegura {  $a.usuarios = A0.usuarios \cup usuarioRegistrado(id, 0, 0, [], 1)$  }
        }
        proc registrarUsuarioConReferidor(in usuario : Usuario, in referidor : Usuario, inout a : AirMiles) {
            requiere {  $referidor \in a.usuarios \wedge a = A0$  }
            asegura {  $a.usuarios = A0.usuarios \cup usuario$  }
            asegura {  $usuario.millasDisponibles = millasDisponibles(usuario, A0) + 500$  }
        }
        proc acumularMillas(in usuario : Usuario, in distancia :  $\mathbb{Z}$ , in nombrePromo : char, inout a : AirMiles) {
            requiere {  $usuario \in a.usuarios \wedge a = A0 \wedge distancia \geq 0 \wedge esPromoActiva(nombrePromo, a)$  }
            asegura { h }
        }
        pred esPromoActiva(nombre : char, a : AirMiles) {  $((\exists p : Promocion) (p \in a.promociones \wedge_L p.nombre = nombre \wedge p.estado = True))$  }
        proc canjearMillas(tipos) : AirMiles {
            requiere { True }
            asegura { algo }
        }
        proc transferirMillas(tipos) : AirMiles {
            requiere { True }
            asegura { algo }
        }
        proc consultarSaldo(tipos) : AirMiles {
            requiere { True }
            asegura { algo }
        }
        proc consultarCategoria(tipos) : AirMiles {
            requiere { True }
            asegura { algo }
        }
        proc rankingMillas(tipos) : AirMiles {

```

```

    requiere { True }
    asegura { algo }
}
proc resumenTransacciones(tipos) : AirMiles {
    requiere { True }
    asegura { algo }
}
proc iniciarPromocion(in nombre : char, in factor :  $\mathbb{Z}$ , inout a : AirMiles) {
    requiere {  $f \geq 1 \wedge a = A0$  }
    asegura {  $(\exists p : Promocion) (p \in a.promociones \wedge p.nombre = nombre \wedge p.factor = factor \wedge p.estado = True)$  }
    asegura {  $(\forall p : Promocion) (p \in A0.promociones \wedge p.nombre \neq nombre \rightarrow p \in a.promociones)$  }
    asegura { }
}
proc iniciarPromocionConTope(tipos) : AirMiles {
    requiere { True }
    asegura { algo }
}
proc finalizarPromocion(tipos) : AirMiles {
    requiere { True }
    asegura { algo }
}
proc elMasAcumulador(tipos) : AirMiles {
    requiere { True }
    asegura { algo }
}
proc elMasCanjeador(tipos) : AirMiles {
    requiere { True }
    asegura { algo }
}
proc paresTransferidoresExclusivos(tipos) : AirMiles {
    requiere { True }
    asegura { algo }
}
aux usuarioRegistrado(in id :  $\mathbb{Z}$ , in millasTotales :  $\mathbb{Z}$ , in millasDisponibles :  $\mathbb{Z}$ , in historial : seq()) : Usuario = (id, millasTotales, millasDisponibles, historial)
aux millasDisponibles(in u : Usuario, in a : AirMiles) :  $\mathbb{Z} = u.millasDisponibles$ 
aux millasTotales(in u : Usuario, in a : AirMiles) :  $\mathbb{Z} = u.millasTotales$ 
aux historialTransacciones(in u : Usuario, in a : AirMiles) :  $\mathbb{Z} = u.historialTransacciones$ 

aux nombrePromocion(in p : Promocion, in a : AirMiles) :  $\mathbb{Z} = p.nombre$ 
aux factorPromocion(in p : Promocion, in a : AirMiles) :  $\mathbb{Z} = p.factor$  aux estadoPromocion(in p : Promocion, in a : AirMiles) :  $\mathbb{Z} = p.estado$ 
pred existeUsuarioEnSistema(id :  $\mathbb{Z}$ , a : AirMiles) {  $(\exists u_0 : Usuario) (u_0 \in a.usuarios \wedge_L u_0.ID = id)$  }
}

```